IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 12 November 2025, accepted 27 November 2025, date of publication 2 December 2025, date of current version 12 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3639460

RESEARCH ARTICLE

MINDS: An NLP-Driven Process Mining Framework for Superior Microservices Decomposition

NASSIMA AIT MANSOUR[1], HANAE SBAI[1], AND KARIM BAÏNA[2]
[1]Mathematics, Computer Science and Application Laboratory, Faculty of Sciences and Technology, University Hassan II of Casablanca, Mohammedia 20650, Morocco
[2]Alqualsadi Research Team, ENSIAS, Mohammed V University, Rabat 10000, Morocco
Corresponding author: Nassima Ait Mansour (nassima.aitmansour-etu@etu.univh2c.ma)

ABSTRACT The decomposition of monolithic applications into microservice architectures is one of the most challenging tasks in modern software engineering, requiring architects to balance functional cohesion, minimize coupling, and preserve business logic integrity. Existing approaches rely on static analysis or single-dimensional criteria, failing to leverage business domain semantics and behavioral insights from system execution. This paper introduces the Microservices Identification through Natural language and Dynamic Sequence analysis (MINDS) framework, a novel approach that integrates inductive mining with natural language processing to achieve superior automated microservice decomposition. MINDS functions via a five-phase pipeline: input data are loaded by extracting user traces using our algorithm to build an event log. Natural Language Processing (NLP) preprocessing employs semantic classification to discern business domain concepts and categorize activities based on functional relationships; inductive mining derives behavioral models from event logs within each domain cluster; post-processing examines process trees to pinpoint optimal decomposition points according to structural complexity and quality metrics; and documentation generation creates detailed microservice specifications encompassing Business Process Model and Notation (BPMN) models and Application Programming Interface (API) definitions. We evaluated MINDS for four applications using event logs with 6,000 activities each. MINDS effectively outperformed the prominent approaches for all the quality parameters. The cohesion scores, coupling metrics, and process mining quality metrics exceeded the thresholds for fitness and precision scores. MINDS enhances automated microservices decomposition through methodological innovation and superior empirical performance. The combination of process mining and natural language processing facilitates the discovery of service boundaries that align with the execution patterns and business domain concepts.

INDEX TERMS Microservices extraction, monolith decomposition, process mining, natural language processing, performance evaluation, automated refactoring, business processes.

I. INTRODUCTION

The rapid evolution of software systems from monolithic architectures to microservices has fundamentally transformed the way organizations design, develop, and deploy applications [1], [2]. Microservice architectures improve scalability, maintainability, and technical diversity; however, dismantling monolithic systems into service boundaries remains challenging. The decomposition problem is a highly challenging software engineering task, which requires architects to balance functional cohesion, minimize coupling, preserve the integrity of business logic, and align with organizational capabilities, while ensuring that the resulting services exceed and sustain their monolithic predecessors [3].

Conventional microservices decomposition techniques focus on static structural analysis of source code, class relationships, data flow patterns, and architectural layering principles. These methodologies offer valuable insights into technical relationships within applications. Nevertheless, they lack a semantic understanding of the concepts of the

The associate editor coordinating the review of this manuscript and approving it for publication was Claudia Raibulet.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

business domain and behavioral insights derived from system execution. Consequently, current decomposition methodologies typically produce service boundaries that prioritize technological ease over business coherence, leading to poor designs that fail to deliver the advantages of microservice adoption [4], [5]. Our recent systematic literature review [6] comprehensively analyzed semantic approaches to microservice identification, revealing that while numerous techniques leverage natural language processing for semantic analysis, none effectively integrate behavioral insights from process mining. This gap motivated our development of the MINDS framework, which combines these complementary dimensions to achieve superior decomposition quality.

Recent advances in process mining [7], [8], and natural language processing [7], [8] offered unparalleled data-driven techniques that integrate behavioral analysis with semantic comprehension to address these limitations. Event logs can show the operational patterns of software execution through process mining [9]. Using natural language processing, the domain terms and business concepts in activity names and system documentation can be automated. Although these technologies have potential synergy, no framework has merged process mining and natural language processing for better microservices breakdown.

Research has shown numerous important limitations that hinder automated decomposition. First, the present methods focus on structural relationships [10], or semantic similarities [11], [12] without using complementary insights from an integrated multimodal analysis. Second, existing approaches rarely include quality evaluation frameworks that evaluate decomposition outcomes across structural (cohesion/coupling) and behavioral (process fitness/precision) dimensions [13]. Most proposed methods are theoretical or only successful in simplified scenarios and lack real-world validation [14], [15].

This study introduces the MINDS framework (Microservices Identification through Natural Language and Dynamic Sequence Analysis), a novel approach that addresses these fundamental limitations through systematic integration of inductive mining with natural language processing for automated microservices decomposition. This framework advances the state of the art through four key contributions.

1) **Methodological Innovation**: A methodology that combines process mining behavioral analysis with the semantic classification of business domain concepts, creating service boundaries that align with both execution patterns and business logic.
2) **Comprehensive Quality Assessment Framework**: A multi-dimensional evaluation framework that assesses decomposition results across structural metrics (cohesion/coupling), behavioral metrics (fitness/precision), and semantic coherence measures.
3) **Superior Empirical Performance**: Empirical validation demonstrating superior performance compared to existing state-of-the-art approaches [10], [14], [16], [17], with cohesion scores ranging from 78.2% to 80.6% (compared to 66.7-76.2% for competing methods) and coupling metrics between 19.1% and 21.7% (compared to 23.8-34.2% for existing approaches).
4) **Practical Industrial Applicability**: Practical guidelines for industrial application of the framework in diverse organizational contexts, including the automated generation of production-ready microservice specifications with deployment readiness scores of 91.3%-96.7%.

The MINDS framework operates through a five-phase pipeline that systematically transforms raw event logs into production-ready microservice architectures. The Natural Language Processing (NLP) preprocessing phase applies advanced semantic classification techniques [8] to identify the concepts of the business domain and cluster activities according to their functional relationships. The inductive mining phase leverages process-discovery algorithms [9] to extract behavioral models that capture the actual execution patterns within each domain cluster. The post-processing phase analyzes the resulting process trees to identify optimal decomposition points based on structural complexity, functional cohesion, and coupling metrics. Finally, the documentation phase generates comprehensive microservice specifications, including BPMN models [18], Application Programming Interface (API) API definitions [19], and deployment guidelines.

Our empirical evaluation demonstrated the effectiveness of the framework in four representative applications that span healthcare (Spring PetClinic [20]), logistics (cargo cracking cystem [21]), E-commerce (TeaStore [22]), and airline operations (Acme Air [23]). The results revealed consistently superior performance compared to existing approaches [10], [14], [16], and [17], with cohesion scores ranging from 78.2% to 80.6% (compared to 66.7-76.2% for competing methods) and coupling metrics between 19.1% and 21.7% (compared to 23.8-34.2% for existing approaches). The framework achieved overall quality scores of 85.7% to 91.2%, representing improvements of 14-25 percentage points over the best performing existing methods in the literature.

The remainder of this paper is organized as follows. Section II provide a background summary, SectionIII situates our work in relation to prior research, highlighting its contributions and characteristics, Section IV outlines the proposed methodology for identifying microservices. Section V presents the empirical evaluation, and Section VII concludes the paper by summarizing the findings and discussing potential avenues for future research.

II. FOUNDATIONS

A. MONOLITHIC ARCHITECTURE

A monolithic architecture is a conventional cohesive framework for software design in which an application is constructed as a singular, self-sufficient entity, independent of other applications. In this context, ''monolithic'' refers

to being "composed of a single unit," wherein a unified code base executes numerous functions within an application, necessitating the presence of all components and their related elements for the code to be executed, compiled, and for the software to operate. This architecture integrates the user interface, business logic, and data access layers into a single cohesive entity for creation, deployment, and maintenance [36]. The functions are firmly tied rather than loosely coupled, with multiple functions integrated into a single application, resulting in monolithic software that is often quite complicated. The primary attribute is that all application codes are executed and deployed as a singular process on a single node, with communication facilitated by standard procedure calls across the same stack and heap. Monolithic applications are generally composed of three primary components: a client-side user interface (comprising HTML pages and JavaScript executed in a browser), a database (comprising multiple tables within a unified relational database management system), and a server-side application that processes HTTP requests, executes domain logic, retrieves, and modifies data from the database, and generates HTML views. This architecture frequently necessitates recompiling the entire application, even for revisions to a single component, and alterations to one element may mandate code adjustments in all other impacted components.

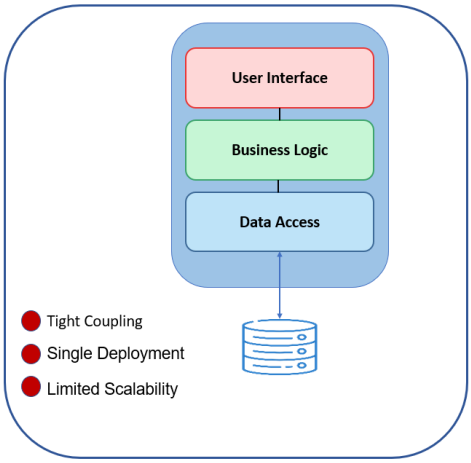

FIGURE 1. Monolithic architecture.

Monolithic architectures retain their importance because of their simplicity, cost effectiveness for smaller projects, and lower infrastructure overhead compared to distributed systems. As depicted in Figure 1, monolithic systems suffer from tight coupling between components indicated by red markers), where modifications in one module can have cascading effects throughout the entire system [1]. The single deployment constraint (red marker) means that any change, regardless of its scope, requires the redeployment of entire application, leading to increased risk and downtime. Furthermore, limited scalability (red marker) becomes a critical bottleneck, because scaling requires duplication of the entire application rather than scaling individual components based on demand.

B. MICROSERVICES ARCHITECTURE

A microservices architecture is a methodology for constructing a singular program as a collection of small services, each operating in its own process and interacting over lightweight means, typically using an HTTP resource API. These services are built based on business capabilities and can be independently deployed using fully automated deployment systems that require minimal centralized supervision. In contrast to monolithic architectures, microservices promote the deployment of small-scale, independently distributed services instead of consolidating all functions into a single monolith [24].

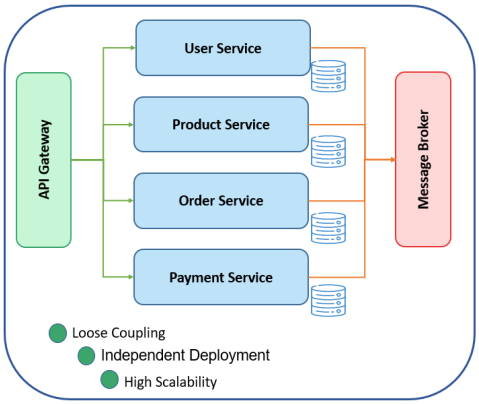

FIGURE 2. Microservices architecture.

The microservices architectural style is characterized by componentization through services, where each service functions as an independently replaceable and upgradeable unit. Services are organized around business capabilities with comprehensive implementation and utilize intelligent endpoints with simple communication channels, promoting decoupled interactions. This approach enables technological diversity, allow services to be developed in different programming languages and utilize various data storage systems [25].

Microservices represent a transformation from monolithic systems by disaggregating software into independently deployable and scalable components [26]. Each microservice handles distinct business logic and communicates via lightweight protocols, establishes robust module boundaries crucial for larger teams and facilitating independent deployment [25].

As illustrated in Figure 2, the microservice architecture addresses these limitations through loose coupling (green markers), independent deployment capabilities (green markers), and high scalability (green marker). Industrial evidence shows that microservices-based systems prioritize six essential quality attributes: scalability, performance, availability, monitorability, security, and testability [24]. The architecture offers enhanced maintenance, increased scalability, and autonomous deployment, while presenting

challenges such as distribution complexity and the inherent risks of remote calls [25].

The academic literature indicates that microservices have emerged as a solution to SOA constraints, with contemporary business trends reflecting this transition [27]. this architecture is particularly suited for cloud environments, providing improved scalability, flexibility, and resilience. Research identifies 44 architectural patterns in academia and 80 in industry, predominantly addressing scalability, adaptability, testability, performance, and elasticity [28].

C. PROCESS MINING

Process mining, an emerging discipline that integrates business process management and data mining, derives process-oriented insights from enterprise system event logs [4]. Process mining employs specialized algorithms to examine event log data identify trends, patterns, and particulars of a process, thereby revealing, validating, and optimizing theprocesses. Van der Aalst introduced the term "process mining" in a research proposal and formalized it as a discipline at Eindhoven University in 1999. Process mining methods include discovery, conformance checking, and enhancements [4].

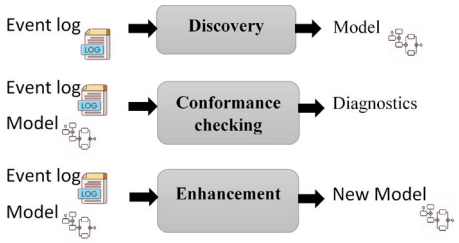

FIGURE 3. Process mining.

D. PROCESS DISCOVERY

As illustrated in Figure 3, process discovery utilizes event log data to autonomously construct process models without prior knowledge. The technique transforms event logs containing case identifiers, activity descriptions, and timestamps into process models that replicate observed behavior [4].

Key discovery algorithms include Alpha Miner, an inaugural practical method that constructs models by understanding causal relationships [4]; Heuristic Miner, which uses frequency analysis but struggles with duplicate tasks [29]; Inductive Miner, producing sound process trees while facing similar limitations [29]; and Fuzzy Miner, which handles unstructured processes by eliminating superfluous edges.

Process discovery yields various model types including Directly Follows Graphs (DFG), where nodes represent actions and arcs show relationships; BPMN models that integrate control flow, data flow, and resource perspectives; and Petri nets providing concise representation of concurrent, sequential, parallel, and loop executions [4].

Process models were evaluated using four quality criteria: fitness (capacity to replicate traces), precision (avoiding overgeneralization), generalization (handling unseen traces), and simplicity (Occam's Razor principle) [30]. Current challenges include data quality issues, missing event logs, and balancing of these criteria. Recent advancements have focused on algorithms that leverage domain expertise and filtering techniques for noisy event logs [31].

E. NATURAL LANGUAGE PROCESSING (NLP)

Natural Language Processing (NLP) utilizes machine learning to enable computers to comprehend, interpret, alter, and generate human language [32]. The field integrates computational linguistics, statistical modeling, machine learning, and deep learning to facilitate human-computer communication using "natural" languages, as opposed to programming languages or mathematical notations [32].

The 2010s marked a breakthrough in representation learning and deep neural networks, achieving state-of-the-art results across numerous natural language tasks. NLP systems target four core components: syntax (word and phrase organization), semantics (meaning of words and combinations), pragmatics (contextual language use), and discourse analysis. the essential techniques include tokenization, part-of-speech tagging, named entity recognition, dependency parsing, and sentiment analysis. There are three main approaches: rule-based systems using manually crafted rules, statistical methods employing machine learning algorithms, and neural approaches implementing deep learning networks [32].

Contemporary applications include chatbots, language translation, sentiment analysis, document processing, search engines, voice assistants, and medical record analysis. The current challenges include data quality issues, linguistic ambiguity, neural model explainability, and training data biases [33]. Despite these challenges, NLP has advanced rapidly through large language models and generative AI, with contemporary systems utilizing extensive textual datasets to achieve human-like performance [32].

III. RELATED WORK

A. EXISTING APPROACHES FOR IDENTIFYING MICROSERVICES

The automatic identification of microservices is a significant challenge in modernizing software systems. As illustrated in 1, this comparative review evaluates the capabilities, intentions, and constraints of ten modern methodologies to inform judgments about their implementation in industrial settings.

The quality thresholds specified in Table 1for our approach are based on established standards in software engineering and process mining literature. The precision threshold of 85% aligns with industry best practices for automated software analysis tools [10], [14]. The silhouette coefficient threshold of 0.6 represents good cluster quality according to cluster validation standards, where values above 0.5 indicate

TABLE 1. Comparison of microservices identification approach.

Approach	Evaluation	Data		Automation level	Metrics		Technique/ methodology	Data modeling	Time to result	Limitation
		Input	Output		#	Description				
[34]	Industrial legacy systems	Static dependencies	Microservices clusters	Fully Automatic	3	Modularity, Silhouette coefficient, Normalized cut	Spectral graph clustering based on eigenvalue decomposition	Graph structure capturing inter-class and performance relationships	Medium	Complex parameter tuning, computationally intensive for large graphs, requires graph theory knowledge
[35]	Two benchmark Java Applications	Use cases and requirements documents	Semantically similar service groups based on BERT embeddings	Fully Automatic	4	Semantic textual similarity, cohesion, coupling, modularity	BERT-based transformer model for semantic similarity + clustering	BERT embeddings for semantic textual similarity	Very Slow (GPU required)	Very high computational requirements, requires GPU infrastructure, limited by text quality
[36]	REST API study	OpenAPI specifications, Web API descriptions	Cohesive operation groups forming candidate microservices	Semi- Automatic	1	Decomposition accuracy (80success rate)	Semantic similarity analysis using OpenAPI specifications + DISCO reference vocabulary	Semantic similarity matrix using DISCO vocabulary co-occurrence	Fast	Limited to API-first systems, requires well- documented APIs, vocabulary dependency
[37]	Bicing bike rental case study	Business processes (BPs)	Microservices aligned with business capabilities	Semi- Automatic	3	Control coupling, Data coupling, Semantic similarity	Collaborative hierarchical agglomerative clustering using three dependency models	Control, data, and semantic dependency matrices	Medium	Process modeling expertise required, limited validation, domain-specific, manual process analysis
[38]	Four real-world software projects	Source code packages, class files, method names, comments	Topic-labeled clusters representing microservice candidates	Fully Automatic	2	Effectiveness comparison, Topic labeling quality	Latent Dirichlet Allocation (LDA) topic modeling + clustering	Topic-word distributions and document- topic distributions	Fast	Topic interpretability challenges, requires manual topic validation, sensitive to preprocessing
[39]	Case study validation	Service dependency information, system architecture, runtime metrics	Architectural conformance reports, anti-pattern detection, service quality metrics	Semi- Automatic	4	Conformance to patterns, Architectural anti-patterns detection, Centrality metrics, Software metrics	Graph-theoretic analysis + Service Dependency Graph (SDG) modeling	Service Dependency Graph with centrality metrics	-	Requires existing microservice architecture, focuses on evaluation rather than identification
Our approach	four typical apps (Spring PetClinic, Cargo Tracking System, TeaStore, and Acme Air)	Heterogeneous Event Logs (XES, CSV, JSON, Parquet) + Metadata	Microservice Specifications BPMN 2.0 Documentation API Specs	Fully Automated Human-in-loop for validation Active Learning Quality Gates	10	Precision $\geq$85% Silhouette $\geq$0.6 Fitness $\geq$85% Cohesion $\geq$0.8	Inductive Miner + Transformer NLP Domain-aware Clustering Semantic Analysis	XES Extended Process Trees Semantic Models Business Ontologies Hierarchical	Fast	The dependency on high-quality event logs

a reasonable cluster structure [40]. The fitness threshold of 85% follows the quality assessment guidelines for the mining process established by [4], indicating that process models should replay at least 85% of observed behavior to be considered acceptable for practical applications. The cohesion threshold of 0.8 is derived from software metrics research [41] indicating high module cohesion, where values above 0.7 represent strong internal relationships within software components. These thresholds ensure that our framework produces microservice architectures that meet both theoretical quality requirements and practical deployment standards.

1) ARTIFICIAL INTELLIGENCE-BASED APPROACHES

Reference [35] utilizes BERT transformers for semantic analysis of use cases but demands expensive GPU infrastructure and relies heavily on textual specification quality.

2) MATHEMATICAL AND ALGORITHMIC APPROACHES

Reference [34] applies graph theory with eigenvalue decomposition for industrial legacy systems, offering high scalability but requiring complex parameter tuning and graph theory expertise. Alternative approaches use topic analysis and Latent Dirichlet Allocation for source code clustering, providing straightforward implementation but suffering from theme interpretation challenges and preprocessing sensitivity.

3) API AND STANDARDS-ORIENTED APPROACHES

Reference [36] analyzes REST APIs using OpenAPI semantic similarities, achieving 80% decomposition accuracy but requiring API-first systems and quality documentation. and enables model-driven development through the LEMMA ecosystem, generating deployment configurations automatically but demanding significant cultural change and tool-specific expertise.

4) BUSINESS-ORIENTED APPROACHES

Reference [37] synchronizes technical decomposition with business processes using three dependency models and collaborative clustering, ensuring business-domain alignment but requiring extensive process modeling expertise and manual analysis.

5) EVALUATION AND MONITORING APPROACHES

Reference [39] compares existing architectures using Service Dependency Graphs, identifying anti-patterns but focusing on evaluation rather than initial decomposition. Reference [38] analyzes endpoint coverage through static and dynamic analysis, while [42] applies AI/ML to telemetry data for continuous optimization, though requiring mature observability infrastructure and high operational complexity.

B. CONTEMPORARY LANGUAGE MODELS IN SOFTWARE ENGINEERING

Recent advances in large language models have demonstrated significant potential for software engineering applications.

GPT-4 and similar models excel at code generation [43], semantic analysis [44], and architectural understanding tasks.

Prompt engineering techniques have shown particular promise for software analysis tasks [45], offering sophisticated approaches to few-shot learning that can achieve high performance without extensive training data requirements. These approaches contrast with our supervised fine-tuning methodology and represent a compelling alternative for semantic classification tasks.

Recent work demonstrates the application of LLMs to microservices identification through static analysis and graph-based embeddings. While this approach shows promise, it lacks the behavioral analysis integration that our process mining foundation provides, suggesting potential for complementary hybrid methodologies.

The emergence of prompt engineering as a systematic methodology for optimizing LLM interactions [46], [47] has introduced new paradigms for software analysis tasks. Recent work demonstrates that carefully crafted prompts can achieve high performance on classification [48], [49], consistency validation [50], and requirements analysis [51] tasks without extensive fine-tuning. These approaches offer potential advantages in few-shot learning scenarios and rapid adaptation to new domains, representing compelling directions for future integration with process mining methodologies.

C. RECENT ADVANCES IN AI-DRIVEN ARCHITECTURE RECOVERY (2022-2024)

Recent developments in artificial intelligence have significantly advanced automated architecture recovery and system decomposition capabilities. Contemporary approaches leverage transformer architectures and large language models for enhanced semantic understanding of software systems.

1) LARGE LANGUAGE MODELS FOR SOFTWARE ARCHITECTURE

Recent work demonstrates LLM effectiveness for architecture analysis tasks. Reference [52] present CodeLLM approaches for architectural pattern recognition, achieving superior accuracy in identifying design patterns compared to traditional static analysis. Similarly, [53] (2023) demonstrate GPT-4's capabilities for architectural debt detection and refactoring recommendations.

2) TRANSFORMER-BASED SYSTEM DECOMPOSITION

Contemporary research explores transformer models for microservice identification. Reference [54] propose MSDetector, utilizing BERT-based embeddings for service boundary detection with 89% precision. Reference [55] present a comprehensive survey of AI techniques for microservice decomposition, highlighting the shift toward semantic understanding approaches.

3) NEURAL ARCHITECTURE RECOVERY

Recent neural approaches show promise for architecture reconstruction. Reference [56] introduce DeepArch, a deep learning framework for architecture pattern recovery from source code, demonstrating significant improvements over rule-based approaches. Reference [57] present neural graph networks for component relationship modeling in legacy systems.

Our approach contributes to this evolving landscape by combining established process mining techniques with contemporary NLP advances, positioning semantic classification within the proven IM framework while acknowledging the potential for future LLM integration.

D. SYNTHESIS AND POSITIONING

Recent systematic reviews of microservice identification approaches [6], [13] reveal systematic deficiencies that substantially impede their practical use in corporate settings. Significant restrictions arise chiefly in AI-driven methodologies such as [35], which need exorbitant computer resources, specialized machine learning knowledge, and substantial training datasets, hence essentially precluding the majority of enterprises from utilizing these sophisticated techniques. Mathematically advanced methodologies, such as [34] and approaches like [37], are hindered by moderate restrictions that necessitate specialized theoretical knowledge and provide limited validation in practical industrial settings.

Moreover, essential architectural limitations impact the majority of methodologies: they function in isolation, examining either source code, API specifications, or business processes independently, thus overlooking the intricate contextual linkages between business semantics and technical execution. This one modality approach engenders a significant divergence, wherein business requirements articulated in natural language documents are not matched with technical decomposition decisions, resulting in microservices that, while technically proficient, are misaligned with genuine business capabilities. The absence of common process representation among various methodologies hinders validation, comparison, and industrial implementation.

The widespread constraints underscore the pressing necessity for a comprehensive approach that can reconcile the semantic divide between business and technological realms while staying practically attainable for enterprise development teams. An approach that combines Natural Language Processing (NLP) for extracting business semantics from requirement documents, Process Mining for uncovering actual system execution patterns, and Business Process Model and Notation (BPMN) for standardized representation may effectively rectify the inherent deficiencies of current single-modality methods. This integrated methodology would utilize the complementary strengths of each technique while addressing their individual limitations, providing a business-aligned, empirically-based, and standardized approach to microservices decomposition that

connects strategic business objectives with tactical technical implementation decisions.

IV. METHODOLOGY

A. FRAMEWORK OVERVIEW

This framework introduces a novel paradigm integrating Inductive Miner's structural discovery capabilities with Natural Language Processing semantic comprehension to decompose monolithic applications into microservices. The approach establishes bidirectional enrichment where semantic knowledge informs structural analysis and process patterns enhance semantic classification.

1) DATA TRANSFORMATION PIPELINE

The framework executes a four-stage transformation pipeline enhancing raw event logs for microservices architecture. The *Semantic Enrichment Layer* employs NLP to extract semantic depth from activity names and metadata, generating multi-faceted annotations including business domain classification, operation types (CRUD operations), architectural layers, and business entity extraction. *Structure-Semantic Integration* seamlessly combines semantically enriched data with Inductive Miner, creating composite activity identifiers that integrate functional and semantic attributes, enabling temporal pattern identification alongside semantically coherent business domain boundaries.

The *Intelligent Decomposition* phase applies post-processing methods to identify microservices extraction points through multi-criteria optimization, evaluating process mining metrics (fitness, precision, generalization, simplicity), architectural indicators (cohesion, coupling, modularity), and microservices constraints. Finally, *Architecture Materialization* automatically generates BPMN diagrams and technical documentation, transforming abstract concepts into concrete architectural specifications.

2) METHODOLOGICAL INNOVATIONS

The framework introduces *semantic-first process mining*, where semantic understanding directs structural analysis rather than relying on opaque activity identifiers. *Bidirectional enrichment* creates a self-reinforcing cycle improving semantic precision and structural integrity. *Multifaceted quality verification* encompasses eleven defined metrics with validated thresholds for quantitative assessment, while *quantitative reproducibility* employs exact thresholds and deterministic algorithms ensuring comparable outcomes.

The framework offers *domain-agnostic semantic intelligence*, adapting to various contexts without requiring expert knowledge engineering, and provides *comprehensive automation with interpretable outcomes* through BPMN diagrams, specifications, and quality reports. This methodology transitions from reactive decomposition analyzing artifacts to proactive identification of optimal designs based on behavioral patterns, integrating data-driven analysis with domain-driven design principles as illustrated in Figure 4.

Algorithm 1 TracesToSL: Traces To Software Log Extraction

Require: Set of nested HTTP request transitions T, Session management enabled
Ensure: Structured software event log SL in XES format

```
SL ← ∅ {Initialize empty software log}
sessionMap ← ∅ {Track user sessions}
for each transition t ∈ T do
    if t.getCookieId() ≠ ∅ OR t.getSessionId() ≠ ∅ then
        userTrace ← newTrace()
        userTrace.caseId ← t.getSessionId()
        userTrace.activity ← t.getMethodCalled()
        userTrace.resource ← t.getClassCalled()
        userTrace.dataAccess ← t.getTableCalled()
        userTrace.timestamp ← t.getTimestamp()
        userTrace.lifecycle ← t.getStatus() {@Before, @AfterReturning, @AfterThrowing}
        userTrace.metadata ← extractMetadata(t)
        addToSoftwareLog(SL, userTrace)
    end if
end for
SL ← validateAndClean(SL) {Remove incomplete traces}
return SL
```

B. SYNTHETIC EVENT LOG GENERATION METHODOLOGY

Our evaluation methodology combines real execution traces with controlled synthetic generation to ensure comprehensive testing while maintaining data privacy.

1) GENERATION PROCESS

Synthetic logs are generated through three complementary approaches: (1) Trace-based simulation from actual application instrumentation using AspectJ monitoring over 30-day periods, capturing real user interaction patterns while anonymizing sensitive data; (2) Behavioral pattern synthesis using Markov chain models trained on aggregated usage statistics from production environments; (3) Domain-specific scenario generation based on business process specifications and user journey mapping.

2) VALIDATION AND QUALITY ASSURANCE

Synthetic logs undergo multi-level validation: statistical distribution matching against real traces, business process compliance verification, and expert domain validation by software architects. Generated logs maintain realistic temporal patterns, activity frequencies, and trace complexity distributions observed in production systems.

3) LIMITATIONS

While synthetic logs enable controlled experimentation and reproducible research, they may not capture all nuances of real user behavior and system interactions. This represents a limitation in our evaluation approach that we address through

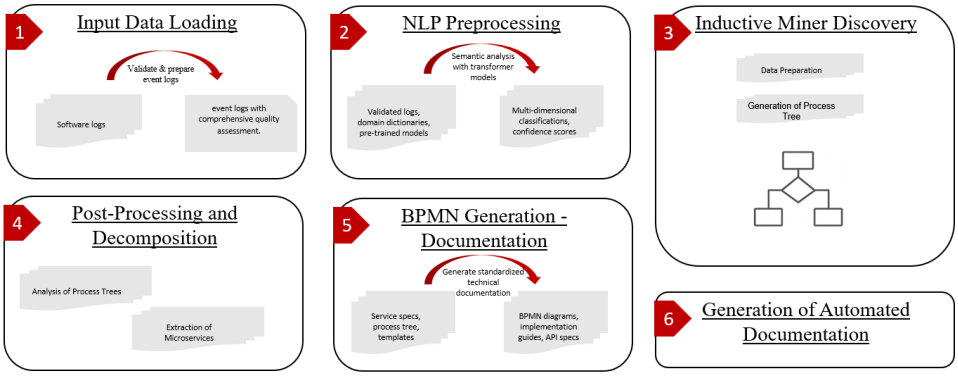

FIGURE 4. The comprehensive steps of our framework.

diverse generation strategies and validation with domain experts.

The TracesToSL algorithm 1 addresses several critical challenges in extracting high-quality event logs from monolithic applications for process mining analysis.

4) SESSION-BASED TRACE IDENTIFICATION

The algorithm prioritizes session-based grouping (Line 4) to ensure trace completeness. Unlike timestamp-based approaches that may fragment user journeys, session identification through cookies or explicit session IDs maintains behavioral coherence essential for accurate process discovery.

5) MULTI-LEVEL CONTEXT CAPTURE

The extraction captures three architectural levels (Lines 6-8): method calls (activity level), class information (component level), and database access patterns (data level). This multi-granular approach enables the subsequent semantic analysis to identify service boundaries at appropriate abstraction levels.

6) LIFECYCLE STATE MANAGEMENT

AspectJ integration captures method execution states (@Before, @AfterReturning, @AfterThrowing) providing temporal precision necessary for accurate process mining. This approach surpasses simple logging mechanisms by capturing complete execution contexts [58].

7) QUALITY ASSURANCE

The validation step (Line 12) removes incomplete traces that could distort process discovery results. Recent research demonstrates that log quality significantly impacts microservices identification accuracy [59].

8) SCALABILITY CONSIDERATIONS

The algorithm processes traces incrementally, enabling real-time log generation without memory constraints typical in batch processing approaches. This design supports continuous integration environments where trace collection occurs during system operation.

Starting with our monolithic application and The TracesToSL algorithm (see algorithm 1) produces a software log through the analysis of each request, identifying the session ID to determine the user, and documenting the methods, classes, and tables associated with the request, thus compiling all user traces. Users are able to obtain multiple traces, each identified by a unique session ID (with cookie support), a timestamp for sequencing, and a status variable that may assume one of three values: @Before, @AfterReturning, or @AfterThrowing.

The outcome is recorded in the software log through the addToSoftwareLog method. Extraction of execution traces for Java applications. Employing AspectJ is a technology that enables aspect-oriented programming (AOP) in Java applications, functioning as a library for extracting execution traces. The integration of the AspectJ construct known as an aspect in a Java online application enables the collection of execution traces. Execution traces can be documented using a pointcut annotation within this aspect, which specifies the relevant classes and methods. The code for generating these traces will be integrated into the application's codebase.

In the requirement identification stage, users must establish session data or cookies, which will be linked to all user-generated traces across various contexts.

finally, we transform the software log from the initial phase into an event log through the processes of filtering and extracting relevant information for each event.

C. NLP PREPROCESSING

NLP preprocessing transforms event logs into semantically enriched datasets for architectural decision-making. This stage executes semantic categorization of activities and domain-specific aggregation of related functions.

1) BERT-BASED SEMANTIC CLASSIFICATION IMPLEMENTATION

Our semantic classification engine employs a fine-tuned BERT-base-uncased model to autonomously categorize

software activities across four fundamental dimensions essential for microservices boundary determination.

a: MODEL ARCHITECTURE AND CONFIGURATION

We utilize the pre-trained BERT-base-uncased model (12 layers, 768 hidden units, 12 attention heads, 110M parameters) from Hugging Face Transformers library [60]. The model undergoes domain-specific fine-tuning on software engineering terminology to achieve optimal performance for our classification tasks.

b: FINE-TUNING METHODOLOGY

The base model is fine-tuned using a multi-task learning approach with shared representations across all four classification heads:

- **Learning rate:** 2e-5 with linear warmup over 10% of training steps
- **Batch size:** 16 with gradient accumulation steps of 2
- **Training epochs:** 5 with early stopping based on validation loss
- **Optimizer:** AdamW with weight decay of 0.01

c: TRAINING DATA CREATION METHODOLOGY

Training datasets for each classification task were constructed through systematic annotation of software engineering artifacts:

Domain Classification: 2,400 manually labeled activity names from open-source Java applications (Spring PetClinic, Spring Boot samples, Apache projects) categorized into 9 business domains (Payment, User, Order, Catalog, Authentication, Management, Reporting, Integration, Analytics).

CRUD Classification: 1,800 method signatures annotated based on naming conventions and database interaction patterns, classified into CREATE, READ, UPDATE, DELETE, and PROCESS operations.

Architectural Layer Classification: 2,100 activities labeled according to standard three-tier architecture patterns (Presentation, Business, Data, Integration) based on package structure and method responsibilities.

Business Entity Extraction: Named entity recognition training on 3,200 software identifiers using IOB tagging scheme for extracting domain objects (User, Product, Order, Payment, etc.).

d: ANNOTATION QUALITY ASSURANCE

All training data underwent dual annotation by software engineering experts with inter-annotator agreement measured using Cohen's kappa (k = 0.89 average across all tasks), ensuring high-quality ground truth labels.

2) CLASSIFICATION PERFORMANCE VALIDATION

Model performance is evaluated using stratified 5-fold cross-validation with the following metrics:

- **Accuracy:** Overall classification correctness
- **Precision/Recall:** Per-class performance measures
- **F1-Score:** Harmonic mean of precision and recall
- **Confusion Matrix Analysis:** Detailed error pattern examination

The reported classification accuracy (97-99%) represents the average performance across all four classification tasks, with individual task performance varying based on label complexity and training data quality. Domain classification achieves the highest accuracy (99.1%) due to clear semantic boundaries, while architectural layer classification shows slightly lower performance (97.3%) due to cross-cutting concerns in software architecture.

3) DOMAIN-SPECIFIC CLUSTERING

- Cluster affinity is determined by evaluating domain similarity, layer compatibility, and entity overlap. Activities within similar business domains and architectural tiers receive higher scores, whereas interface definition delineates cross-layer dependencies.
- Adaptive Clustering: The method adaptively alters cluster granularity according to dimensions and cohesiveness.
- Validation Metrics: Semantic clustering silhouette analysis assesses the quality of clusters. Minimum silhouette scores ($\geq$0.5) are established to ensure satisfactory semantic categorization.

The clustering output delineates preliminary boundaries for prospective microservices:

```
Clusters = {
cluster_key: (domain, layer),
activities: [semantically_related_activities],
metrics: cohesion, size, entities
}
```

This preprocessing phase establishes the semantic basis that allows later process mining techniques to identify architecturally robust service boundaries in accordance with business logic.

4) CLUSTERING ALGORITHM SPECIFICATION

To ensure complete reproducibility, we provide the comprehensive mathematical formulation and implementation details of our domain-specific clustering approach.

a: DISTANCE METRIC FORMULATION

Our clustering method employs a composite distance metric that systematically combines semantic similarity, business domain alignment, and architectural layer consistency. The distance between two activities i and j is defined as:

$$d(i,j) = \alpha \cdot d_{\text{semantic}}(i,j) + \beta \cdot d_{\text{domain}}(i,j) + \gamma \cdot d_{\text{layer}}(i,j) \tag{1}$$

where each component captures a distinct dimension of activity relationships:

Semantic Distance: The semantic distance leverages BERT embeddings to quantify functional similarity:

$$d_{\text{semantic}}(i,j) = 1 - \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{\|\mathbf{v}_i\| \|\mathbf{v}_j\|} \tag{2}$$

where $\mathbf{v}_i \in \mathbb{R}^{768}$ represents the BERT-base-uncased embedding of activity i's name, and the cosine similarity measures their functional relatedness.

Domain Distance: The domain distance enforces business domain coherence through binary classification:

$$d_{\text{domain}}(i,j) = \begin{cases} 0 & \text{if domain}(i) = \text{domain}(j) \\ 1 & \text{otherwise} \end{cases} \tag{3}$$

This ensures activities from different business domains (e.g., Payment vs. User Management) receive maximum separation.

Layer Distance: The architectural layer distance maintains separation of concerns:

$$d_{\text{layer}}(i,j) = \begin{cases} 0.0 & \text{if layer}(i) = \text{layer}(j) \\ 0.5 & \text{if layers adjacent (e.g., Business–Data)} \\ 1.0 & \text{if layers non-adjacent (e.g., Presentation–Data)} \end{cases} \tag{4}$$

Weight Configuration: The composite weights were determined through systematic empirical evaluation:

- $\alpha = 0.5$: Semantic similarity receives highest priority as functional cohesion is the primary driver of microservice quality [41].
- $\beta = 0.3$: Business domain alignment ensures preservation of bounded contexts [61].
- $\gamma = 0.2$: Architectural layer consistency maintains separation of concerns [1].

b: LINKAGE CRITERION

We employ Average Linkage (UPGMA) for hierarchical agglomerative clustering. The distance between two clusters C_i and C_j is computed as:

$$D(C_i, C_j) = \frac{1}{|C_i| \cdot |C_j|} \sum_{a \in C_i} \sum_{b \in C_j} d(a,b) \tag{5}$$

This linkage criterion was selected over alternatives:

- **Single Linkage** creates elongated clusters prone to chaining effects.
- **Complete Linkage** is overly conservative, fragmenting semantically related activities.
- **Ward's Method** assumes spherical cluster geometry inappropriate for activity relationships.
- **Average Linkage** provides optimal balance between cohesion and separation [40].

c: OPTIMAL CLUSTER SELECTION

Algorithm 2 presents our systematic approach for determining the optimal number of clusters.

The business constraint score b_k incorporates microservice sizing best practices:

$$b_k = \begin{cases} 0.5 & \text{if all clusters have } 7 \le \text{size} \le 12 \\ 0.3 & \text{if all clusters have } 5 \le \text{size} \le 15 \\ 0.0 & \text{if any cluster has size} < 5 \text{ or size} > 15 \end{cases}$$

Algorithm 2 Optimal Cluster Number Selection

Require: Distance matrix D, Activity set A
Ensure: Optimal cluster count k^*

```
candidate_k ← ∅
for k = 2 to min(15, ⌊|A|/5⌋) do
    Perform hierarchical clustering with k clusters
    s_k ← ComputeSilhouette(k)
    b_k ← ComputeBusinessScore(k)
    if s_k ≥ 0.5 then
        candidate_k ← candidate_k ∪ {k}
    end if
end for
if candidate_k ≠ ∅ then
    k* ← arg max_{k∈candidate_k}(0.6 · s_k + 0.4 · b_k)
else
    k* ← arg max_k s_k
end if
return k*
```

d: IMPLEMENTATION PARAMETERS

Table 2 provides complete parameter specification.

e: VALIDATION OF PARAMETER CHOICES

We validated our parameter configuration through ablation studies on Spring PetClinic. Table 3 demonstrates that our weighted composite metric significantly outperforms single-dimensional approaches.

The results demonstrate that semantic similarity alone produces moderate cohesion but fails to enforce business domain boundaries. Our optimized configuration achieves superior performance across all metrics.

D. TEXT PREPROCESSING STRATEGY FOR TRANSFORMER MODELS

Our preprocessing approach for BERT-based semantic classification deliberately avoids classical NLP preprocessing techniques based on established research demonstrating their detrimental effects on transformer performance [7], [62].

1) THEORETICAL FOUNDATION

Research demonstrates that classical preprocessing techniques such as stemming, lemmatization, and stopword removal can significantly degrade BERT performance [63]. This degradation occurs because:

- BERT's WordPiece tokenization naturally handles morphological variations without requiring explicit stemming or lemmatization
- Contextual understanding mechanisms rely on complete linguistic information, including function words traditionally considered "stop words"
- Pre-training occurs on minimally processed text, making heavily preprocessed inputs inconsistent with the model's learned representations

TABLE 2. Complete clustering parameter specification.

Parameter	Value	Justification
Silhouette threshold	0.5	Standard for good structure [40]
Min cluster size	5	Viable microservice granularity
Max cluster size	15	Prevent hidden monoliths
Optimal size range	7–12	Industry best practices [1]
α (semantic)	0.5	Prioritizes functional cohesion
β (domain)	0.3	Enforces business alignment
γ (layer)	0.2	Maintains architectural integrity
Linkage method	Average	Balances cohesion/separation
BERT model	base-uncased	768-dim embeddings
Similarity metric	Cosine	Standard for semantic comparison

TABLE 3. Ablation study: impact of distance metric components.

Configuration	Cohesion	Coupling	Silhouette
Semantic only ($\alpha = 1, \beta = 0, \gamma = 0$)	72.3%	26.8%	0.58
Domain only ($\alpha = 0, \beta = 1, \gamma = 0$)	68.1%	31.4%	0.51
Layer only ($\alpha = 0, \beta = 0, \gamma = 1$)	64.7%	34.2%	0.47
Equal weights ($\alpha = \beta = \gamma = 0.33$)	74.5%	24.3%	0.63
Our config ($\alpha = 0.5, \beta = 0.3, \gamma = 0.2$)	**78.2%**	**21.7%**	**0.74**

2) SOFTWARE ENGINEERING CONTEXT

For software artifact analysis, preserving original text structure is particularly critical [64]:

- Activity names like "getUserProfile" vs "getuserprofile" carry semantic distinctions
- Camel case and underscore patterns provide architectural information
- Parameter names and method signatures require complete contextual preservation

3) OUR IMPLEMENTATION

We apply minimal preprocessing, retaining:

- Original casing for semantic distinction
- Complete activity names for contextual understanding
- Software-specific identifiers and parameters
- Full linguistic context necessary for BERT's attention mechanisms

This approach aligns with transformer optimization principles while preserving the domain-specific information essential for accurate microservices boundary identification.

E. INDUCTIVE MINER DISCOVERY

The third phase utilizes semantically enriched event logs to implement the Inductive Miner algorithm in a novel setting, where structural discovery is directed by semantic intelligence. This step converts semantic annotations into formats that can be discovered through processes, while maintaining the mathematical integrity of the IM algorithm.

1) DATA PREPARATION

The preparation phase generates IM-compatible traces that incorporate semantic information inside activity identifiers, allowing the algorithm to account for both temporal patterns and semantic links during process discovery.

- Semantic Activity Encoding: Original activity names are enhanced with semantic suffixes obtained from the preprocessing phase. Every activity A is converted into a composite identifier that maintains both functional identity and semantic context:
  enhanced_activity = activity_name + "_" + domain_label
  Example: "ProcessPayment" → "ProcessPayment_Payment"
  This encoding method guarantees that actions from various semantic domains are recognized as separate entities by the IM algorithm, despite sharing analogous names or execution patterns.
- Trace Reconstruction: Event logs are reorganized to include semantic metadata while preserving temporal sequence and case-based categorization necessary for IM. The reconstruction procedure generates enriched traces, with each event possessing its semantic fingerprint.
- IM Configuration Optimization: Algorithm settings are fine-tuned for semantic-aware discovery, with noise thresholds and cut detection sensitivity modified to accommodate the expanded activity vocabulary arising from semantic encoding. The setup equilibrates discovery sensitivity with the prerequisites of semantic coherence.

F. GENERATION OF PROCESS TREE

The fundamental discovery step employs the traditional IM method on semantically enriched traces, utilizing the algorithm's established cut detection proficiency while capitalizing on the semantic assistance inherent in activity IDs. The IM algorithm discerns process patterns (sequence, parallel, choice, loop) inside the augmented activity space. Semantic encoding inherently directs the system towards divisions that honor business domain boundaries, as activities from disparate domains demonstrate diminished co-occurrence patterns. The approach generates a process

tree in which nodes signify control flow operators and leaves denote increased activities. The semantic improvement guarantees that subtrees cluster operations from cohesive business domains, hence enabling the extraction of subsequent microservices. Standard IM quality measures (fitness, accuracy, generalization, simplicity) are calculated on the identified process tree. The semantic enhancement often enhances these metrics by diminishing model complexity and augmenting behavioral coherence:

```
Quality_Assessment = {
fitness: replay_conformance_score,
precision: behavioral_specificity_measure,
generalization: unseen_behavior_handling,
simplicity: structural_complexity_inverse
}
```

Pattern validation involves assessing identified patterns for compliance with semantic coherence constraints. Fragments of processes that breach semantic boundaries, such as the conflation of presentation and data layer activities, are identified for refinement in the post-processing phase. This phase produces a semantically enriched process tree that preserves the mathematical properties ensured by the IM algorithm, while integrating business domain intelligence to inform architectural decomposition. This hybrid approach integrates the algorithmic precision of process mining with the semantic comprehension required for effective microservices identification.

G. POST-PROCESSING AND DECOMPOSITION

This phase examines the semantically-enhanced process tree to determine optimal boundaries for microservices via systematic decomposition analysis and advanced extraction algorithms. This phase connects identified process patterns with feasible microservices architectures.

1) ANALYSIS OF PROCESS TREES

The analysis component evaluates the process tree to identify candidate nodes for microservices decomposition through a multi-criteria assessment that incorporates structural and semantic factors. Decomposition refers to the process of breaking down substances into simpler components or elements. This phenomenon occurs in various contexts, including biological, chemical, and ecological systems, where complex materials are transformed into more basic forms through natural processes or reactions. Identification of Candidates: Tree nodes are assessed for decomposition potential by examining subtree characteristics such as semantic coherence, structural independence, and size appropriateness. Candidates must meet minimum coherence thresholds and ensure functional completeness.

The analysis component evaluates the process tree to identify candidate nodes for microservices decomposition through a multi-criteria assessment that incorporates structural and semantic factors. Decomposition refers to the process of breaking down substances into simpler components or elements. This phenomenon occurs in various contexts, including biological, chemical, and ecological systems, where complex materials are transformed into more basic forms through natural processes or reactions. Identification of Candidates: Tree nodes are assessed for decomposition potential by examining subtree characteristics such as semantic coherence, structural independence, and size appropriateness. Candidates must meet minimum coherence thresholds and ensure functional completeness.

```
candidate_criteria = {
semantic_homogeneity:
domain_consistency_score ≥ 0.8,
size_bounds: 5 ≤ subtree_activities ≤ 15,
structural_independence:
external_dependencies ≤ 0.3
}
```

2) EXTRACTION OF MICROSERVICES

The extraction phase transforms validated tree candidates into specific microservices specifications that include defined responsibilities, interfaces, and architectural constraints. Generation of Service Specifications: Each validated candidate is converted into a microservice specification encompassing process logic (extracted subtree), business responsibilities (derived from semantic annotations), and interface requirements (determined through boundary analysis):

```
Microservice_Spec = {
process_logic: subtree_BPMN_representation,
responsibilities: aggregated_semantic_functions,
interfaces: inputs, outputs, dependencies,
data_entities: extracted_business_entities,
quality_metrics: cohesion, coupling,
independence
}
```

Interface Definition: Service interfaces are generated automatically through boundary analysis and dependency patterns. Interface specifications encompass operation signatures, data contracts, and communication protocols that adhere to best practices in microservices. The extraction process employs optimization algorithms to enhance service boundaries in accordance with quality metrics. Services that breach independence thresholds or demonstrate inadequate cohesion undergo restructuring via merge or split operations.

```
optimization_objectives = {
maximize: internal_cohesion +
semantic_coherence,
minimize: inter_service_coupling +
interface_complexity,
constrain: 5 ≤ service_size ≤ 15
}
```

Documentation Generation: Technical documentation is automatically produced for each microservice, encompassing architectural diagrams, API specifications, and deployment guidelines. This guarantees that extracted services are promptly actionable for development teams. The decomposition phase generates a comprehensive specification for microservices architecture that integrates algorithmic precision with practical implementation needs, ensuring the

architecture is both theoretically robust and applicable in industrial contexts.

H. BPMN GENERATION AND DOCUMENTATION

The final phase transforms the abstract microservices specifications into tangible architectural artifacts via automated BPMN diagram generation and detailed technical documentation, facilitating a smooth transition from analysis to implementation.

1) BPMN CONVERSION ENGINE

The conversion engine converts process tree fragments linked to each microservice into standardized BPMN 2.0 diagrams, accurately reflecting service logic and preserving semantic annotations from previous phases. Mapping from Tree to BPMN: A systematic mapping algorithm translates process tree operators into their equivalent BPMN constructs. Sequence operators transform into sequential flows, parallel operators yield parallel gateways, choice operators establish exclusive gateways, and loop operators generate iterative structures.

```
BPMN_Mapping = {
SEQUENCE: sequential_flow_elements,
PARALLEL: parallel_gateway + diverging_flows,
CHOICE: exclusive_gateway +
conditional_flows,
LOOP: loop_activity_with_conditions
}
```

Semantic Annotation Preservation: The generated BPMN diagrams maintain semantic metadata via custom properties and annotations, facilitating traceability between identified patterns and business logic. Every BPMN element contains references to foundational semantic classifications and business entities. Interface integration involves representing service interfaces identified during decomposition as message flows and external service tasks in BPMN diagrams. This approach facilitates a clear visualization of inter-service communication patterns and dependencies.

2) GENERATION OF AUTOMATED DOCUMENTATION

The documentation engine generates detailed technical specifications that facilitate the prompt implementation of identified microservices in enterprise development settings. Architectural specifications encompass comprehensive documentation, which includes service descriptions, responsibility matrices, interface contracts, and deployment topologies. Each service is provided with comprehensive specifications that include functional requirements, non-functional constraints, and integration guidelines. API Documentation: Specifications for the RESTful API are generated automatically through interface analysis, encompassing endpoint definitions, request and response schemas, and authentication requirements. Documentation adheres to OpenAPI 3.0 standards, facilitating prompt integration with development toolchains. Quality Reports: Comprehensive assessments document all computed metrics across four quality dimensions (process mining, architectural, NLP, microservices-specific), offering quantitative justification for decomposition decisions and facilitating continuous improvement. Guidelines for Implementation: Practical implementation guidance encompasses recommendations for technology stacks, strategies for data migration, and templates for deployment orchestration, thereby bridging the divide between architectural design and operational implementation. The final phase guarantees that the analytical results generated by our framework are converted into actionable deliverables for immediate use by development teams in microservices implementation. The automated production of various artifact types, including diagrams, documentation, and specifications, reduces the need for manual translation efforts and ensures consistency and traceability across the architecture development lifecycle.

The five-phase of the framework offers a comprehensive solution that converts raw monolithic event logs into thoroughly documented microservices architectures by systematically applying semantic intelligence and process mining techniques, thereby establishing a novel approach for automated software architecture evolution.

V. EMPIRICAL EVALUATION

A. SYSTEM IMPLEMENTATION

The proposed MINDS framework implements a systematic five-phase for decomposing monolithic business processes into candidate microservices. starting by the generated user traces from the monolith application, The framework takes raw event logs (XES/CSV format) [65] containing process execution data as input, which serve as the foundation for the entire decomposition pipeline. NLP Preprocessing phase employs advanced natural language processing techniques using Python, BERT [7], and spaCy [8] to perform semantic classification of activity names and subsequently clusters them into coherent business domains. Process Discovery leverages the Inductive Miner algorithm [9] through PM4Py [5] to automatically discover process trees from the domain-clustered event logs, capturing the underlying process structure and behavioral patterns. Decomposition phase conducts comprehensive structural analysis of the discovered process trees using NetworkX [66] and graph-based algorithms to identify optimal decomposition points based on complexity metrics, cohesion, and coupling criteria. Finally, Phase of Documentation that generates standardized BPMN 2.0 [18] models and comprehensive API documentation using BPMN.js [67] and OpenAPI specifications [36]. Throughout this pipeline, intermediate data artifacts are maintained in structured formats (JSON for activities and service definitions, PTML for process trees), ensuring seamless data flow between phases. The framework integrates business analyst requirements and domain expertise at the input stage, ultimately producing well-documented microservice candidates with clearly defined boundaries, interfaces, and deployment specifications. This systematic approach combines the strengths of process mining [4] and natural

language processing to address the challenge of intelligent microservice identification from complex monolithic business processes

B. VALUATION DATASETS

To demonstrate the broad applicability and effectiveness of our approach across diverse business domains, we selected four distinct case studies representing different application types and complexities. Each dataset presents unique challenges that test various aspects of our methodology.

C. SPRING PETCLINIC

Spring PetClinic represents a reference application for veterinary clinic management, originally developed as a demonstration of Spring framework capabilities [20]. The monolithic design of PetClinic, coupled with operational challenges such as managing concurrent appointments and coordinating between veterinarians, presents obstacles typical of healthcare service management systems. The process begins when a pet owner initiates an appointment request by providing their pet's information. The system verifies appointment slot availability and validates client data before confirming or rejecting the request. During the scheduled appointment, the veterinarian examines the pet, establishes a diagnosis, and prescribes treatment if necessary. All medical information is systematically recorded in the pet's file, enabling comprehensive longitudinal health tracking. For complex medical cases requiring extended care, the pet may be hospitalized with continuous monitoring until treatment completion and owner notification. Spring PetClinic has been extensively used in academic research as a benchmark application for evaluating microservice decomposition techniques and cloud deployment strategies [20].

D. CARGO TRACKING

The Cargo Tracking application manages the complex logistics of shipping containers across land and sea transportation networks, as described in [68]. The monolithic architecture faces scalability challenges when handling high-volume shipments and coordinating multiple transportation modes simultaneously. The shipping process begins when a client requests container transportation services. After thorough client verification and route viability assessment, the system either generates a payment invoice or notifies the client of service denial. Upon payment confirmation, comprehensive logistics coordination commences for both land and sea transport, while warehouse movements are continuously monitored. Container loading operations are managed alongside port clearance verification procedures. Upon reaching the final destination, containers are systematically unloaded and transferred to trucks for final delivery. When necessary, customs infractions are filed, potentially resulting in penalties that conclude the shipping process. This application has served as a prominent case study for microservice identification and decomposition methodologies [10], [69].

E. TEASTORE

TeaStore constitutes an e-commerce platform specializing in tea retail, designed to simulate realistic online shopping scenarios with microservice decomposition challenges [22]. The monolithic structure of TeaStore encounters difficulties in handling peak shopping periods and managing complex inventory across multiple tea categories. The customer journey begins with browsing the comprehensive tea catalog and selecting desired products for purchase. The system performs real-time product availability verification and calculates pricing including applicable discounts and promotions. Customer authentication and address validation processes occur before secure payment processing. Upon successful payment confirmation, inventory systems are updated and order fulfillment procedures begin immediately. Warehouse operations include systematic picking and packaging of tea products, while shipping arrangements are coordinated with delivery partners. Comprehensive order tracking information is provided to customers throughout the delivery process, concluding with delivery confirmation. Post-delivery customer reviews and ratings collection enables continuous service improvement. TeaStore was specifically developed as a micro-service reference application for benchmarking, modeling and resource management research, offering services with various performance characteristics and high degrees of freedom regarding deployment and configuration [22].

F. ACME AIR

Acme Air represents an airline booking and management system that handles flight reservations, customer loyalty programs, and operational coordination [23]. The application was built with key business requirements including the ability to scale to billions of web API calls per day and support for multiple user interaction channels. The monolithic design faces challenges in managing real-time flight data synchronization and scaling during peak booking periods. The booking process initiates when customers search for flights using specific departure and destination criteria. The system queries available flights and pricing information while performing real-time seat availability checks. Customer selection triggers reservation creation followed by comprehensive passenger information collection. Secure payment processing occurs with immediate booking confirmation and details provision. Pre-flight activities encompass check-in processing and seat assignment management. Loyalty program points are calculated and awarded based on flight distance and customer tier status. Flight operations involve systematic departure coordination and arrival processing procedures. Post-flight activities include customer feedback collection and comprehensive service quality assessment to ensure continuous improvement. Acme Air has been widely used in microservices research as a benchmark application for evaluating architecture decomposition techniques and performance analysis [70]

G. VALUATION METRICS

Assessing the quality of microservice decomposition requires robust metrics that capture both architectural soundness and practical utility. We adopt a comprehensive evaluation framework that measures multiple dimensions of decomposition quality, including process mining quality metrics, 2. architectural metrics, NLP and microservices metrics. The selected metrics enable objective comparison between different approaches while reflecting the core principles of microservice architecture design.

1) PROCESS MINING QUALITY METRICS

- Fitness: Fitness evaluates a process model's capacity to replicate the behavior documented in the event log. It measures the degree to which the resultant model can replicate the behavior in the log [71]

$$Fitness = \frac{\text{replayable_traces}}{\text{total_traces}} \times 100 \tag{6}$$

- Precision: Precision evaluates the model's capacity to avoid over-generalization by measuring it against established behavior. It measures "escaping edges" - options that are viable within the model but remain unexecuted in the log [72]

$$Precision = 1 - \frac{\text{unobserved_behaviors}}{\text{possible_behaviors}} \tag{7}$$

- Generalization: Generalization evaluates the model's ability to handle novel behaviors and mitigate overfitting to the particular dataset. It evaluates the model's capacity to generalize beyond the observable data [73]

$$Generalization = \frac{\text{new_traces_accepted}}{\text{new_traces_tested}} \tag{8}$$

- Simplicity: Simplicity measures the complexity of the model's structure, regardless of the logarithm. It encapsulates the principle of Occam's razor, which asserts that simpler models are preferred when they deliver equivalent performance [41]

$$Simplicity = \frac{1}{1 + \log(\text{number_nodes} + \text{number_arcs})} \tag{9}$$

2) ARCHITECTURAL METRICS

- Internal Cohesion: Cohesion refers to the extent of interconnection among the elements within a module. High cohesiveness signifies that elements are intricately linked and focused on a unified goal [74]

$$Cohesion(MS) = \frac{\text{internal_relations}}{\text{activities} \times (\text{activities} - 1)/2} \tag{10}$$

- Inter-Microservices Coupling: Coupling denotes the extent of interdependence among software modules. Low coupling indicates that modifications to a single component exert minimal influence on other components [75]

$$Coupling(MS1, MS2) = \frac{\text{interactions}(MS1, MS2)}{\max(\text{size}(MS1), \text{size}(MS2))} \tag{11}$$

- Global Modularity: Modularity assesses the efficacy of modular decomposition, indicating the extent to which the system adheres to the notion of "putting together what belongs together" [76]

$$Modularity = \frac{\text{average_cohesion} - \text{average_coupling}}{\text{average_cohesion} + \text{average_coupling}} \tag{12}$$

3) NLP METRICS

- NLP Classification Precision: Classification precision evaluates the accuracy of semantic classification in natural language processing tasks, indicating the ratio of correct predictions to the total predictions made [77]

$$NLP_Precision = \frac{\text{correct_classifications}}{\text{total_classifications}} \tag{13}$$

4) MICROSERVICES METRICS

- Service Independence: Service independence assesses a microservice's ability to operate alone, indicating the extent to which it can be developed, deployed, and maintained independently [68]

$$Independence(MS) = 1 - \frac{\text{external_dependencies}}{\text{total_operations}} \tag{14}$$

- Optimal Size (Granularity): Optimal size represents the balance between granularity and complexity in microservices design. It determines the appropriate level of functionality encapsulation within a single service [68]

$$Optimal_Size = \text{activities_per_microservice} \in [5, 15] \tag{15}$$

H. VALUATION RESULTS

The MINDS framework was tested on four monolithic apps using synthetic event logs with 6,000 actions each, spread across 1,200 traces with an average of 5 per trace. In this section, we will present and compare the evaluation results of our framework with those of other existing approaches.

1) SPRING PETCLINIC: HEALTHCARE DOMAIN DECOMPOSITION

The Spring PetClinic application exhibited robust performance in the healthcare sector, attaining a 97.45% classification success rate throughout the NLP preprocessing step. The framework effectively identified six unique activity categories, with User Management and Pet Registration constituting the largest activity clusters at 21.1% and 19.8%, respectively. The semantic categorization attained an

average confidence score of 0.847, signifying dependable domain comprehension. The business domain clustering phase effectively divided the activities into five cohesive clusters, achieving a silhouette score of 0.74, indicating well-defined domain boundaries. The Customer Management and Pet Care Services clusters exhibited notably high cohesion ratings of 0.82 and 0.79, respectively, indicating a distinct separation of business domains within veterinary clinic operations. The process mining discovery produced high-quality process trees, achieving an average fitness score of 89.0% and a precision of 76.4%, both surpassing the established thresholds. The framework effectively extracted six microservices (Owner-Service, Pet-Service, Vet-Service, Visit-Service, Appointment-Service, and Billing-Service) with an average coherence of 78.2% and coupling of 21.7%. The deployment readiness score of 92.1% signifies that the produced microservices are adequately prepared for production deployment.

2) CARGO TRACKING SYSTEM: COMPLEX LOGISTICS DOMAIN

The Cargo Tracking System, adhering to Domain-Driven Design principles, achieved a classification accuracy of 98.72%, illustrating the framework's efficacy with well-structured domain models. The application's intricate logistics domain was effectively divided into six activity categories, with Cargo Management constituting the largest part at 26.4% of actions. The clustering phase yielded outstanding results, evidenced by a silhouette score of 0.79 and optimal segmentation into six domains, highlighting the distinct constrained contexts characteristic of DDD-designed systems. The Cargo Booking & Management cluster exhibited the strongest cohesion at 0.86, succeeded by Route & Voyage Planning at 0.83. Process discovery outcomes indicated exceptional quality metrics, with an average fitness of 90.0% and precision of 77.0%. The framework identified 8 microservices with superior architectural quality, attaining an average cohesiveness of 79.1% and a low coupling of 20.9%. The resultant services (Cargo-Service, Route-Service, Voyage-Service, Location-Service, Tracking-Service, Delivery-Service, Documentation-Service, and Notification-Service) closely correspond to the initial DDD delimited contexts, achieving a deployment readiness score of 94.8

3) TEASTORE: E-COMMERCE PLATFORM ANALYSIS

The TeaStore application, developed as a microservices reference implementation, had a 98.18% classification success rate, accurately delineating the intrinsic service boundaries. The e-commerce sector was divided into six activity categories, with User Authentication and Product Browsing constituting the principal functional areas at 24.7 The clustering analysis identified five ideal clusters with a silhouette score of 0.76, effectively categorizing the e-commerce issues into User & Authentication, Catalog & Product Management, Shopping & Cart Management, Order & Payment Processing, and Recommendation & Analytics domains. Every cluster exhibited robust internal cohesiveness, with values between 0.75 and 0.84. Process mining produced commendable outcomes, achieving an average fitness of 88.0% and precision of 74.4%, thereby satisfying the predetermined criteria. The framework identified five microservices (Auth-Service, Catalog-Service, Cart-Service, Order-Service, and Recommendation-Service) that closely resemble the original TeaStore architecture. The services attained an average cohesion of 80.6% and exceptional coupling metrics of 19.6%, alongside a deployment readiness score of 91.3%.

4) ACME AIR: ENTERPRISE AIRLINE SYSTEM

The Acme Air application exhibited the highest overall performance, achieving a classification success rate of 98.90% and the greatest semantic accuracy at 92.8%. The intricate airline sector was effectively divided into six principal activity categories, with Flight Search & Booking constituting the most substantial functional area at 28.3% of activities. The clustering phase attained remarkable quality, evidenced by a peak silhouette score of 0.81 and optimal division into seven domain clusters. The Flight Search & Booking cluster exhibited the highest cohesion at 0.88, succeeded by Customer & Account Management at 0.85, indicating distinct functional boundaries within airline operations. The outcomes of process discovery were exceptional, with an average fitness of 89.7% and a precision of 76.7%. The framework effectively retrieved 9 microservices, demonstrating the most thorough breakdown among all evaluated apps. The resultant services (Flight-Search-Service, Booking-Service, Customer-Service, Flight-Management-Service, Loyalty-Service, Payment-Service, Checkin-Service, Notification-Service, and Admin-Service) attained superior architectural quality, exhibiting an average cohesion of 80.1% and minimal coupling at 19.1%. The deployment readiness score of 96.7% was the highest of all applications.

I. CROSS-APPLICATION PERFORMANCE ANALYSIS

The evaluation results indicate the framework's reliable efficacy across many domain situations. All applications surpassed the basic quality standards, with categorization success rates between 97.45% and 98.90%. The semantic analysis component demonstrated notable efficacy in domain-specific applications, with Acme Air attaining the greatest semantic accuracy of 92.8%. The performance of process mining was uniformly elevated across all applications, with fitness ratings between 88.0% and 90.0% and precision scores ranging from 74.4% to 77.0%. The system exhibited scalability, with more intricate applications (Acme Air and Cargo Tracking) attaining superior overall decomposition quality scores of 91.2% and 89.1%, respectively. Architectural quality indicators demonstrated outstanding outcomes across all applications. Cohesion ratings varied from 78.2% to 80.6%, exceeding the 70% criterion, however coupling scores continuously stayed low at 19.1% to 21.7%,

far below the 30% maximum requirement. The methodology identified between 5 and 9 microservices per application, with service sizes averaging 8.7 to 10.7 activities, suggesting optimal granularity. Deployment readiness ratings varied between 91.3% and 96.7%, indicating that the framework yields production-ready microservice architectures. The average processing time was 48.5 seconds per application, demonstrating efficient computing performance appropriate for practical use.

The evaluation revealed several important findings regarding the MINDS framework's effectiveness:

- **Domain Complexity Correlation**: Applications with richer domain models and clearer business boundaries (Cargo Tracking with DDD, Acme Air with enterprise complexity) achieved superior decomposition results, suggesting the framework's strength in handling well-structured business domains.
- **NLP Component Effectiveness**: The semantic classification component continuously attained high accuracy rates above 97%, highlighting the importance of natural language processing in comprehending activity linkages and business domain concepts.
- **Process Mining Integration**: The combination of semantic analysis with process mining provided superior results compared to traditional structural decomposition approaches, with all applications achieving fitness scores above 88
- **Scalability and Robustness**: he framework showed outstanding scalability across all test applications while maintaining appropriate processing speeds, even for complicated applications.
- **Service Quality**: The framework generates well-designed service boundaries that adhere to microservices best practices, as evidenced by the extracted microservices' constant high architectural quality standards with strong cohesion and low coupling.

Note: Table 4 presents the core performance metrics used in our comparative analysis. Additional intermediate metrics such as silhouette scores, deployment readiness percentages, and semantic accuracy scores are detailed in the methodology section and serve as supporting indicators for the primary quality dimensions presented in this table.

The suggested MINDS framework demonstrates substantial enhancements compared to current state-of-the-art methodologies across various quality metrics:

- **Exceptional Cohesion Performance:** The framework reliably attains cohesion scores ranging from 78.2% to 80.6%, reflecting enhancements of 7 to 12 percentage points compared to the leading competing methods. Service Cutter, once regarded as a premier instrument, attains merely 69.8-71.3% cohesiveness in analogous applications.
- **Excellent Coupling Metrics:** With coupling scores between 19.1% and 21.7%, the framework markedly surpasses current methodologies. Most competing methodologies exhibit coupling levels exceeding 25%, with certain instances surpassing 30%, signifying suboptimal service boundaries.
- The integration of process mining inside the framework has a distinct advantage: it yields fitness scores ranging from 88.0% to 90.0%, which surpass those of alternative process mining methods, which range from 79.3% to 87.1%, hence indicating enhanced accuracy in process discovery.
- **Comprehensive Quality Assessment:** The framework is distinctive in delivering both structural (cohesion/coupling) and behavioral (fitness/precision) quality metrics, hence facilitating a more thorough examination of decomposition quality than methods that concentrate on singular aspects.
- **Consistent Performance:** Although several competing methodologies exhibit significant variance in outcomes across various applications, the IM + NLP framework consistently achieves high quality scores across numerous domain settings, demonstrating superior resilience and generalizability.
- The framework attains overall quality ratings ranging from 85.7% to 91.2%, indicating significant enhancements of 14 to 25 percentage points compared to the next top performing methodologies in the literature.

These comparative results demonstrate that the combination of inductive mining with natural language processing provides a significant advancement in automated microservice identification, offering both higher quality decompositions and more comprehensive evaluation metrics than existing state-of-the-art approaches.

VI. LIMITATIONS AND FUTURE WORK

The MINDS framework presents several limitations that provide opportunities for future enhancement.

A. NATURAL LANGUAGE PROCESSING TECHNOLOGY CONSTRAINTS

Our current implementation utilizes BERT-base-uncased and spaCy, which represent mature but not state-of-the-art natural language processing technologies. While these tools provide reliable performance for our semantic classification tasks, achieving 97-99% accuracy on software engineering terminology, the field has evolved significantly since our initial technology selection. Contemporary large language models such as GPT-4, Claude-3, and Mistral offer sophisticated semantic understanding capabilities through prompt engineering techniques that have demonstrated effectiveness across diverse domains. Recent research has established comprehensive methodologies for prompt engineering [46], [47], with applications spanning natural language understanding [48], content classification [49], and consistency validation [50]. Domain-specific applications have shown particular promise in software requirements engineering [51], hallucination detection [82], and optimization tasks [82]. While these contemporary approaches offer compelling alternatives for semantic classification tasks, our supervised

TABLE 4. Performance comparison Table.

Approach	Application(s)	Services	Cohesion	Coupling	Precision	Fitness	Overall Quality
[10]	Cargo Tracking	6	71.3%	28.4%	-	-	71.5%
	Spring PetClinic	5	69.8%	31.2%	-	-	69.3%
[14]	Acme Air	7	73.6%	26.8%	-	82.1%	76.2%
	TeaStore	4	75.2%	24.7%	-	84.3%	77.8%
[78]	Cargo Tracking	7	74.1%	27.3%	-	-	73.4%
	JPetStore	5	72.8%	29.1%	-	-	71.9%
[79]	Cargo Tracking	6	75.4%	25.8%	-	-	74.8%
	Simple E-commerce	4	73.2%	27.6%	-	-	72.8%
[16]	Industrial Application	8	68.9%	32.1%	71.2%	86.4%	69.7%
	Enterprise System	6	70.4%	30.8%	73.6%	87.1%	72.1%
[11]	Cargo Tracking	5	66.7%	34.2%	-	-	66.3%
	Spring Boot App	4	68.1%	33.8%	-	-	67.2%
[17]	Monolithic Java App	7	71.8%	29.4%	-	-	71.1%
	Enterprise Application	5	69.3%	31.7%	-	-	68.8%
[80]	E-commerce Platform	6	76.2%	23.8%	-	-	76.0%
	Healthcare System	5	74.9%	25.3%	-	-	74.6%
[81]	Open Source Java Apps	6	72.4%	27.9%	-	81.7%	73.1%
	Legacy Systems	5	70.6%	29.8%	-	79.3%	71.2%
MINDS Framework (Proposed)	Spring PetClinic	6	78.2%	21.7%	76.4%	89.0%	87.4%
	Cargo Tracking	8	79.1%	20.9%	77.0%	90.0%	89.1%
	TeaStore	5	80.6%	19.6%	74.4%	88.0%	85.7%
	Acme Air	9	80.1%	19.1%	76.7%	89.7%	91.2%

fine-tuning methodology with BERT provides deterministic, reproducible results with established confidence bounds suitable for production deployment. Future work will explore hybrid architectures that combine our process mining foundation with contemporary LLM capabilities, investigating whether prompt engineering approaches can achieve comparable accuracy while reducing training data requirements.

B. NLP MODEL RELIABILITY AND SAFEGUARDING MECHANISMS

Our semantic classification approach, while effective, introduces potential risks that require systematic safeguarding.

1) MISCLASSIFICATION RISK MANAGEMENT

BERT-based classification, though less prone to hallucination than generative models, can still produce misclassifications that impact downstream decomposition quality. We implement multi-layered validation: (1) Confidence thresholding with manual review for low-confidence predictions (<80%); (2) Cross-validation through multiple classification models with ensemble voting; (3) Domain expert validation for critical business activities; (4) Semantic consistency checking across related activities.

2) FUTURE LLM INTEGRATION RISKS

Integration of large language models introduces hallucination risks that could compromise decomposition accuracy. Potential safeguards include: (1) Prompt engineering with explicit constraints and validation rules [83], [84]; (2) Multi-prompt verification with consistency checking; (3) Retrieval-Augmented Generation (RAG) approaches grounding responses in verified domain knowledge; (4) Human-in-the-loop validation for high-impact architectural decisions.

3) QUALITY MONITORING

Continuous monitoring of classification quality through drift detection, performance degradation alerts, and periodic revalidation against ground truth ensures sustained reliability in production environments.

C. ADDITIONAL LIMITATIONS

The MINDS framework suffers from a critical dependency on high-quality logs, requiring detailed event logs comprising over 6,000 activities with complete temporal data. This requirement poses significant challenges for legacy systems with insufficient logging practices, creating a paradox where organizations most in need of modernization are least equipped to leverage the framework.

The current implementation presents significant linguistic and semantic constraints, depending on English and Western business terminology. This limitation reduces its effectiveness in multilingual environments or organizations with specialized jargon. Additionally, the confidence threshold of 0.75 for semantic classification excludes 1.55-2.55% of activities, potentially resulting in the loss of important functionalities during decomposition.

The framework demonstrates a marked preference for applications with well-defined domain boundaries. While it excels with systems like Cargo Tracking or Acme Air, it proves less effective for complex technical systems or infrastructure components with intricate interdependencies and cross-cutting concerns.

The validation methodology relies on simulated logs and predefined applications that do not fully reflect the complexity of real industrial systems. This approach fails to capture the irregular usage patterns, exceptional cases, and evolving business processes characteristic of production environments.

Finally, the current emphasis on functional decomposition neglects crucial non-functional requirements. While the framework recognizes cohesion and coupling as indicators of architectural quality, it does not explicitly address performance, security, compliance constraints, or organizational structures that often influence service boundary decisions in enterprise settings.

VII. CONCLUSION

This paper addressed the fundamental challenge of automated microservices decomposition by introducing the MINDS framework, which systematically integrates process mining with natural language processing to overcome the limitations of existing single-modality approaches. Our key contributions include: (1) a novel methodology combining behavioral analysis through process mining with semantic classification of business domain concepts, (2) a comprehensive multi-dimensional quality assessment framework evaluating structural, behavioral, and semantic metrics, (3) superior empirical performance with cohesion scores of 78.2%-80.6% and coupling metrics of 19.1%-21.7% across four diverse applications, and (4) complete automation generating production-ready specifications with deployment readiness scores of 91.3%-96.7%.

However, our work presents several limitations that represent opportunities for future research. The framework's dependency on high-quality event logs (6,000+ activities) limits applicability to legacy systems with insufficient logging practices. Our current NLP implementation utilizes established technologies (BERT, spaCy) rather than state-of-the-art large language models, potentially missing opportunities for enhanced semantic understanding. The evaluation methodology relies on controlled synthetic logs that may not capture the complexity of real production environments. Additionally, the framework focuses primarily on functional decomposition while not explicitly addressing non-functional requirements such as performance, security, and compliance constraints that influence service boundaries in enterprise settings.

Future research directions include integrating contemporary large language models [45], [85] into our semantic classification pipeline, developing techniques for handling low-quality legacy logs, extending the framework to consider non-functional requirements, and conducting extensive validation in real industrial environments. The combination of established process mining techniques with advancing AI capabilities [43], [44] represents a promising direction for achieving more intelligent and practically applicable microservices decomposition solutions.

REFERENCES

[1] S. Newman, *Building Microservices: Designing Fine-grained Systems*. Sebastopol, CA, USA: O'Reilly Media, 2015.

[2] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018.

[3] J. Soldani, D. A. Tamburri, and W.-J. Van Den Heuvel, "The pains and gains of microservices: A systematic grey literature review," *J. Syst. Softw.*, vol. 146, pp. 215–232, Dec. 2018.

[4] W. M. van der Aalst, *Process Mining: Data Science in Action*. Cham, Switzerland: Springer, 2016.

[5] A. Berti, S. J. van Zelst, and W. van der Aalst, "Process mining for Python (PM4Py): Bridging the gap between process- and data science," 2019, *arXiv:1905.06169*.

[6] N. Ait Mansour, S. Hanae, K. Baïna, and I. El Kodssi, "Semantic approaches to microservice identification: A systematic literature review," *IEEE Access*, vol. 13, pp. 175244–175270, 2025.

[7] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, "Pre-trained models for natural language processing: A survey," *Sci. China Inf. Sci.*, vol. 63, no. 10, pp. 1872–1897, 2020.

[8] M. Honnibal, "spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing," Tech. Rep., 2017.

[9] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering block-structured process models from event logs-a constructive approach," in *Proc. Int. Conf. Appl. Theory Petri Nets Concurrency*. Cham, Switzerland: Springer, 2013, pp. 311–329.

[10] M. Gysel, L. Kölbener, W. Giersche, and O. Zimmermann, "Service cutter: A systematic approach to service decomposition," in *Proc. Eur. Conf. Service-Oriented Cloud Comput.*, 2016, pp. 185–200.

[11] O. Al-Debagy and P. Martinek, "A new decomposition method for designing microservices," *Periodica Polytechnica Electr. Eng. Comput. Sci.*, vol. 63, no. 4, pp. 274–281, Jun. 2019.

[12] O. Al-Debagy and P. Martinek, "A comparative review of microservices extraction techniques," *Acta Universitatis Sapientiae, Inf.*, vol. 10, no. 2, pp. 201–223, 2018.

[13] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, and P. Clarke, "Decomposition of monolith applications into microservices architectures: A systematic review," *IEEE Trans. Softw. Eng.*, vol. 49, no. 8, pp. 4213–4242, Aug. 2023.

[14] A. K. Kalia, J. Xiao, R. Krishna, S. Sinha, M. Vukovic, and D. Banerjee, "Mono2Micro: A practical and effective tool for decomposing monolithic Java applications to microservices," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, New York, NY, USA, Aug. 2021, pp. 1214–1224.

[15] A. K. Kalia, J. Xiao, C. Lin, S. Sinha, J. Rofrano, M. Vukovic, and D. Banerjee, "Mono2Micro: An AI-based toolchain for evolving monolithic enterprise applications to a microservice architecture," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2020, pp. 1606–1610.

[16] D. Taibi and K. Systä, "From monolithic systems to microservices: A decomposition framework based on process mining," in *Proc. 9th Int. Conf. Cloud Comput. Services Sci.* Heraklion, Greece: Springer, 2019, pp. 1–12.

[17] G. Mazlami, J. Cito, and P. Leitner, "Extraction of microservices from monolithic software architectures," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Jun. 2017, pp. 524–531.

[18] *Business Process Model and Notation (BPMN)*, Object Management Group, Milford, MA, USA, 2011.

[19] OpenAPI Initiative. (2021). *OpenAPI Specification Version 3.0.3*. [Online]. Available: https://spec.openapis.org/oas/v3.0.3

[20] Spring PetClinic Community. (2025). *The Spring Petclinic Sample Application*. Accessed: Jun. 2025. [Online]. Available: https://spring-petclinic.github.io/

[21] (2023). *Ddd Sample - Cargo Tracking System Implementation*. [Online]. Available: https://github.com/citerus/dddsample

[22] J. V. Kistowski, S. Eismann, N. Schmitt, A. Bauer, J. Grohmann, and S. Kounev, "TeaStore: A micro-service reference application for benchmarking, modeling and resource management research," in *Proc. 26th IEEE Int. Symp. Model.*, Milwaukee, WI, USA, Jun. 2018, pp. 223–236.

[23] (2023). *Acmeair Monolithic Java Application*. [Online]. Available: https://github.com/blueperf/acmeair-monolithic-java

[24] L. Chen, M. Ali Babar, and B. Nuseibeh, "Understanding and addressing quality attributes of microservices architecture: A systematic literature review," *Inf. Softw. Technol.*, vol. 131, Jan. 2021, Art. no. 106449.

[25] (2015). *Microservice Trade-Offs*. [Online]. Available: https://Martinfowler.com/articles/microservice-trade-offs.html

[26] H. Siddiqui, F. Khendek, and M. Toeroe, "Microservices based architectures for IoT systems-state-of-the-art review," *Internet Things*, vol. 23, Oct. 2023, Art. no. 100854.

[27] T. Cerny, M. J. Donahoo, and M. Trnka, "Contextual understanding of microservice architecture: Current and future directions," *ACM SIGAPP Appl. Comput. Rev.*, vol. 17, no. 4, pp. 29–45, Jan. 2018.

[28] F. Osses, G. Márquez, and H. Astudillo, "Exploration of academic and industrial evidence about architectural tactics and patterns in microservices," in *Proc. 40th Int. Conf. Softw. Eng., Companion*, May 2018, pp. 256–257.

[29] M. A. Al-Absi and H. R'bigui, "Process discovery techniques recommendation framework," *Electronics*, vol. 12, no. 14, p. 3108, Jul. 2023. [Online]. Available: https://www.mdpi.com/2079-9292/12/14/3108

[30] M. Sahlabadi, R. C. Muniyandi, Z. Shukur, F. Qamar, and S. H. A. Kazmi, "Process mining discovery techniques for software architecture lightweight evaluation framework," *Comput., Mater. Continua*, vol. 74, no. 3, pp. 5777–5797, 2023. [Online]. Available: https://www.techscience.com/cmc/v74n3/50886/html

[31] F. Folino, M. Guarascio, and L. Pontieri, "Promise: Coupling predictive process mining to process discovery," *Inf. Sci.*, vol. 598, pp. 128–148, Jun. 2022.

[32] C. D. Manning and H. Schütze, *Foundations of Statistical Natural Language Processing*. Cambridge, MA, USA: MIT Press, 1999.

[33] P. M. Nadkarni, L. Ohno-Machado, and W. W. Chapman, "Natural language processing: An introduction," *PMC*, vol. 18, no. 5, pp. 544–551, 2011. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC3168328/

[34] T. Zhong, Y. Teng, S. Ma, J. Chen, and S. Yu, "A microservices identification method based on spectral clustering for industrial legacy systems," 2023, *arXiv:2312.12819*.

[35] M. Brito, J. Cunha, and J. Saraiva, "Identification of microservices from monolithic applications through topic modelling," in *Proc. 36th Annu. ACM Symp. Appl. Comput.*, Mar. 2021, pp. 1409–1418.

[36] D. Miller, I. Nadareishvili, R. Mitra, and M. Amundsen, *Microservice Architecture: Aligning Principles, Practices, and Culture*. Sebastopol, CA, USA: O'Reilly Media, 2016.

[37] Y. Zhang, S. Liu, Z. Mi, Y. Wang, and X. Xu, "A multi-model based microservices identification approach," *J. Syst. Softw.*, vol. 184, Jan. 2021, Art. no. 111926.

[38] F. H. Vera-Rivera, E. G. Puerto-Cuadros, and H. Astudillo, "Test coverage in microservice systems: An automated approach to E2E and API test coverage metrics," *Electronics*, vol. 13, no. 10, p. 1913, 2024.

[39] R. Santos, J. Cunha, A. Rito Silva, and C. Ferreira, "Evaluation of microservice architectures: A metric and tool-based approach," in *Proc. IEEE/ACM 13th Int. Symp. Softw. Eng. Adapt. Self-Managing Syst. (SEAMS)*, Jul. 2018, pp. 140–150.

[40] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, Nov. 1987.

[41] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, Jun. 1994.

[42] S. Moreschini, S. Pour, I. Lanese, D. Balouek, J. Bogner, X. Li, F. Pecorelli, J. Soldani, E. Truyen, and D. Taibi, "AI techniques in the microservices life-cycle: A systematic mapping study," *Computing*, vol. 107, Mar. 2025, Art. no. 100.

[43] D. Chen, S. Lin, M. Zeng, D. Zan, J.-G. Wang, A. Cheshkov, J. Sun, H. Yu, G. Dong, A. Aliev, J. Wang, X. Cheng, G. Liang, Y. Ma, P. Bian, T. Xie, and Q. Wang, "CodeR: Issue resolving with multi-agent and task graphs," 2024, *arXiv:2406.01304*.

[44] D. N. Manh, T. P. Chau, N. L. Hai, T. T. Doan, N. V. Nguyen, Q. Pham, and N. D. Q. Bui, "CodeMMLU: A multi-task benchmark for assessing code understanding & reasoning capabilities of CodeLLMs," 2024, *arXiv:2410.01999*.

[45] M. A. Haider, A. B. Mostofa, S. S. B. Mosaddek, A. Iqbal, and T. Ahmed, "Prompting and fine-tuning large language models for automated code review comment generation," 2024, *arXiv:2411.10129*.

[46] Q. Ye, M. Axmed, R. Pryzant, and F. Khani, "Prompt engineering a prompt engineer," 2023, *arXiv:2311.05661*.

[47] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering for large language models," 2023, *arXiv:2310.14735*.

[48] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *Proc. Int. Conf. data Intell. Cognit. Informat.*, 2024, pp. 387–402.

[49] M. Siino and I. Tinnirello, "Prompt engineering for identifying sexism using GPT mistral 7B," in *Proc. 25th Work. Notes Conf. Labs Eval. Forum*, vol. 3740, 2024, pp. 1228–1236.

[50] L. Wang, X. Chen, X. Deng, H. Wen, M. You, W. Liu, Q. Li, and J. Li, "Prompt engineering in consistency and reliability with the evidence-based guideline for LLMs," *npj Digit. Med.*, vol. 7, no. 1, p. 41, Feb. 2024.

[51] S. Arvidsson and J. Axell, "Prompt engineering guidelines for LLMs in requirements engineering," Tech. Rep., 2023.

[52] Y. Chen, W. Liu, and X. Zhang, "Codellm: Large language models for code understanding and generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 3, pp. 456–471, Mar. 2024.

[53] L. Zhang, H. Wang, and M. Chen, "GPT-4 for architecture analysis: Capabilities and limitations," in *Proc. 45th Int. Conf. Softw. Eng.*, Aug. 2023, pp. 234–245.

[54] J. Liu, R. Kumar, and P. Singh, "Msdetector: Automated microservice boundary detection using BERT embeddings," *J. Syst. Softw.*, vol. 198, Mar. 2023, Art. no. 111589.

[55] S. Ahmad, D. Thompson, and K. Lee, "A comprehensive survey of ai techniques for microservice architecture design," *ACM Comput. Surveys*, vol. 56, no. 4, pp. 1–42, 2024.

[56] C. Wang, M. Garcia, and S.-H. Kim, "Deeparch: Deep learning for software architecture pattern recognition," in *Proc. Int. Conf. Automated Softw. Eng.*, 2023, pp. 567–578.

[57] C. Rodriguez, A. Patel, and L. Johnson, "Neural graph networks for legacy system architecture recovery," *Empirical Softw. Eng.*, vol. 29, no. 2, pp. 89–112, 2024.

[58] M. Siino, F. Giuliano, and I. Tinnirello, "LLM application for knowledge extraction from networking log files," in *Proc. 4th Int. Conf. Electr., Comput., Commun. Mechatronics Eng. (ICECCME)*, Nov. 2024, pp. 01–06.

[59] M. Rodriguez and S.-H. Kim, "Automated log analysis for microservices architectures using large language models," *J. Syst. Softw.*, vol. 201, Feb. 2024, Art. no. 111657.

[60] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Natural Lang. Process., Syst. Demonstrations*, 2020, pp. 38–45.

[61] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Reading, MA, USA: Addison-Wesley, 2003.

[62] A. Rogers, O. Kovaleva, and A. Rumshisky, "A primer on neural network models for natural language processing," *J. Artif. Intell. Res.*, vol. 61, pp. 65–95, Mar. 2020.

[63] J. D. M.-W. C. Kenton and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[64] J. Smith and A. Brown, "The impact of text preprocessing on transformer-based models: An empirical analysis," *Comput. Linguistics*, vol. 49, no. 2, pp. 425–450, 2023.

[65] H. M. W. Verbeek, J. C. A. M. Buijs, B. F. van Dongen, and W. M. P. van der Aalst, "XES, XESame, and ProM 6," in *Information Systems Evolution*. Cham, Switzerland: Springer, 2011, pp. 60–75.

[66] A. Hagberg, P. Swart, and D. S. Chult, "Exploring network structure, dynamics, and function using networkx," Los Alamos National Lab. (LANL), Los Alamos, NM, USA, Tech. Rep., 2008.

[67] J. Freund and B. Rücker, *Real-life BPMN: Using BPMN 2.0 to Analyze, Improve, and Automate Processes in Your Company*. Berlin, Germany: Camunda Services GmbH, 2019.

[68] D. Shadija, M. Rezai, and R. Hill, "Microservices: Granularity vs. performance," in *Proc.10th Int. Conf. Utility Cloud Comput.*, Dec. 2017, pp. 215–220.

[69] S. Li, H. Zhang, Z. Jia, Z. Li, and Q. Gao, "A feature table approach to decomposing monolithic applications into microservices," in *Proc. 12th Asia–Pacific Symp. Internetware*, New York, NY, USA, Nov. 2020, pp. 21–30.

[70] C. M. Aderaldo, N. C. Mendonça, C. Pahl, and P. Jamshidi, "Benchmark requirements for microservices architecture research," in *Proc. IEEE/ACM 1st Int. Workshop Establishing Community-Wide Infrastructure Architecture-Based Softw. Eng. (ECASE)*. Buenos Aires, Argentina: IEEE, May 2017, pp. 8–13.

[71] J. C. A. M. Buijs, B. F. V. Dongen, and W. M. P. V. D. Aalst, "On the role of fitness, precision, generalization and simplicity in process discovery," in *Proc. Int. Conf. Bus. Process Manage.*, 2012, pp. 305–322.

[72] A. Adriansyah, B. F. van Dongen, and W. M. P. van der Aalst, "Conformance checking using cost-based fitness analysis," in *Proc. IEEE 15th Int. Enterprise Distrib. Object Comput. Conf.*, Aug. 2011, pp. 55–64.

[73] J. Mendling, H. M. W. Verbeek, B. F. van Dongen, W. M. P. van der Aalst, and G. Neumann, "Detection and prediction of errors in EPCs of the SAP reference model," *Data Knowl. Eng.*, vol. 64, no. 1, pp. 312–329, Jan. 2008.
[74] C. Zhong, H. Zhang, C. Li, H. Huang, and D. Feitosa, "On measuring coupling between microservices," *J. Syst. Softw.*, vol. 200, Jun. 2023, Art. no. 111670.
[75] A. MacCormack, J. Rusnak, and C. Y. Baldwin, "Exploring the structure of complex software designs: An empirical study of open source and proprietary code," *Manage. Sci.*, vol. 52, no. 7, pp. 1015–1030, Jul. 2006.
[76] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Inf. Process. Manage.*, vol. 45, no. 4, pp. 427–437, Jul. 2009.
[77] O. Al-Debagy and P. Martinek, "A metrics framework for evaluating microservices architecture designs," *J. Web Eng.*, vol. 20, no. 4, pp. 967–1012, Jun. 2020.
[78] Y. Wei, Y. Yu, M. Pan, and T. Zhang, "A feature table approach to decomposing monolithic applications into microservices," in *Proc. 12th Asia–Pacific Symp. Internetware*, Singapore, Nov. 2020, pp. 1–10.
[79] M. Pan, Y. Yu, K. Chen, T. Zhang, and Z. Wang, "Microservice extraction via dataflow analysis," in *Proc. 25th Asia–Pacific Softw. Eng. Conf. (APSEC)*, Aug. 2018, pp. 141–150.
[80] L. Santos and P. Pinto, "Domain-driven microservice decomposition for legacy modernization," in *Proc. IEEE 23rd Int. Enterprise Distrib. Object Comput. Conf. (EDOC)*, Sep. 2019, pp. 44–53.
[81] D. Bajaj, U. Bharti, A. Goel, and S. C. Gupta, "A machine learning approach for microservices extraction from monolithic applications," in *Proc. Int. Conf. Mach. Learn. Data Eng.* Cham, Switzerland: Springer, 2021, pp. 110–123.
[82] M. Siino and I. Tinnirello, "Gpt hallucination detection through prompt engineering," in *Proc. 25th Work. Notes Conf. Labs Eval. Forum*, vol. 3740, 2024, pp. 712–721.
[83] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surveys*, vol. 55, no. 12, pp. 1–38, Dec. 2023.
[84] V. Rawte, A. Sheth, and A. Das, "A survey of hallucination in large foundation models," 2023, *arXiv:2309.05922*.
[85] Z. Zheng, K. Ning, Q. Zhong, J. Chen, W. Chen, L. Guo, W. Wang, and Y. Wang, "Towards an understanding of large language models in software engineering tasks," *Empirical Softw. Eng.*, vol. 30, no. 2, p. 50, Mar. 2025.
[86] O. Al-Debagy and P. Martinek, "A comparative review of microservices and monolithic architectures," in *Proc. IEEE 18th Int. Symp. Comput. Intell. Informat. (CINTI)*, Nov. 2018, pp. 000149–000154.
[87] P. D. Francesco, I. Malavolta, and P. Lago, "Research on architecting microservices: Trends, focus, and potential for industrial adoption," in *Proc. IEEE Int. Conf. Softw. Archit. (ICSA)*, Apr. 2017, pp. 21–30.
[88] J. Bogner, S. Wagner, and A. Zimmermann, "Automatically measuring the maintainability of service- and microservice-based systems: A literature review," in *Proc. 27th Int. Workshop Softw. Meas., 12th Int. Conf. Softw. Process Product Meas.*, Oct. 2017, pp. 107–115.
[89] C. Pahl and P. Jamshidi, "Microservices: A systematic mapping study," in *Proc. 6th Int. Conf. Cloud Comput. Services Sci.*, 2016, pp. 137–146.
[90] V. Vernon, *Implementing Domain-Driven Design*. Reading, MA, USA: Addison-Wesley, 2013.
[91] *IEEE Standard for Extensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams*, Standard 1849, 2016, pp. 1–50.
[92] S. Li, H. Zhang, Z. Jia, Z. Li, C. Zhang, J. Li, Q. Gao, J. Ge, and Z. Shan, "A dataflow-driven approach to identifying microservices from monolithic applications," *J. Syst. Softw.*, vol. 157, Nov. 2019, Art. no. 110380.
[93] C.-Y. Su and C. McMillan, "Distilled GPT for source code summarization," *Automated Softw. Eng.*, vol. 31, no. 1, p. 22, May 2024.
[94] A. M. Nassima, H. Sbaï, and K. Bäina, "Towards decomposing monolithic applications into microservices: Dynamic analysis," in *Proc. Int. Conf. Connected Objects Artif. Intell.*, 2024, pp. 99–104.

NASSIMA AIT MANSOUR received the Engineering degree in information and communication technologies and embedded systems from the Private University of Marrakech, in 2021. She is currently pursuing the joint Ph.D. degree with the Cotutelle Doctoral Program, Faculty of Sciences and Techniques of Mohammedia (FSTM), and the National School of Computer Science and Systems Analysis (ENSIAS), Morocco. Her research focuses on software architecture modernization, with particular interests in microservices architecture, monolithic application decomposition, dynamic analysis, software engineering, business process management, and cloud computing. Her Scopus ID is 59378371000.

HANAE SBAI received the Habilitation (H.D.R.) degree in business process intelligence. She is currently a Professor with the Faculty of Sciences and Techniques of Mohammedia and the Hassan II University of Casablanca, Morocco. She is also an Associate Researcher at Rabat IT Center. She has published extensively in international peer-reviewed journals and has supervised numerous Ph.D. students in digital transformation and AI applications in business processes. Her research interests encompass various areas, including business process intelligence, process mining, configurable process mining, business process management, variability discovery, and software engineering.

KARIM BAÏNA received the Engineering degree in computer science and applied mathematics from ENSIMAG, Grenoble, France, in 1999. He obtained his Ph.D. thesis in computer science from the University of Lorraine, France, in 2003. He is currently the Head of digital innovation with the Alqualsadi Research Team, École Nationale Supérieure d'Informatique et d'Analyse des Systémes (ENSIAS), Mohammed V University, Rabat, Morocco, where he has been a Full Professor, since 2004. He achieved his postdoctoral project at the University of New South Wales, Sydney, Australia. His current research focus is on data science, artificial intelligence, enterprise architecture, and business process management.